DBMS LEVEL 3 MINI PROJECT — WEEK 4 SUBMISSION

ScholarMesh

Semantic Research-Notes Discovery & Study-Group Matchmaking Platform

Course	DBMS Experiential Learning — Level 3 Project
Institution	PES University, Electronic City Campus
Department	Computer Science and Engineering
Submitted by	Darshan | SRN: PES2UG24CS143
Week 4 Task	Project title · ER diagram · Relational schema · DDL · Execution

1. Project Title & Problem Statement

Title

ScholarMesh — Semantic Research-Notes Discovery & Study-Group Matchmaking Platform

Problem Statement

Students accumulate large volumes of notes, PDFs, and reference material across a semester, but existing storage tools (Drive folders, WhatsApp groups, local files) only support exact filename or keyword search. A student who needs "notes that explain attention mechanisms" gets no results unless that exact phrase was used in a filename. Separately, students look for study partners with an ad-hoc method — asking around in class WhatsApp groups — with no way to see who is actually engaging with the same topics.

ScholarMesh addresses both problems together: it lets students upload notes/papers, semantically search across all of them by meaning (not just keywords), and automatically suggests / helps form study groups based on shared courses and overlapping topic interest — inferred from what students search for and upload.

Why this goes beyond CRUD

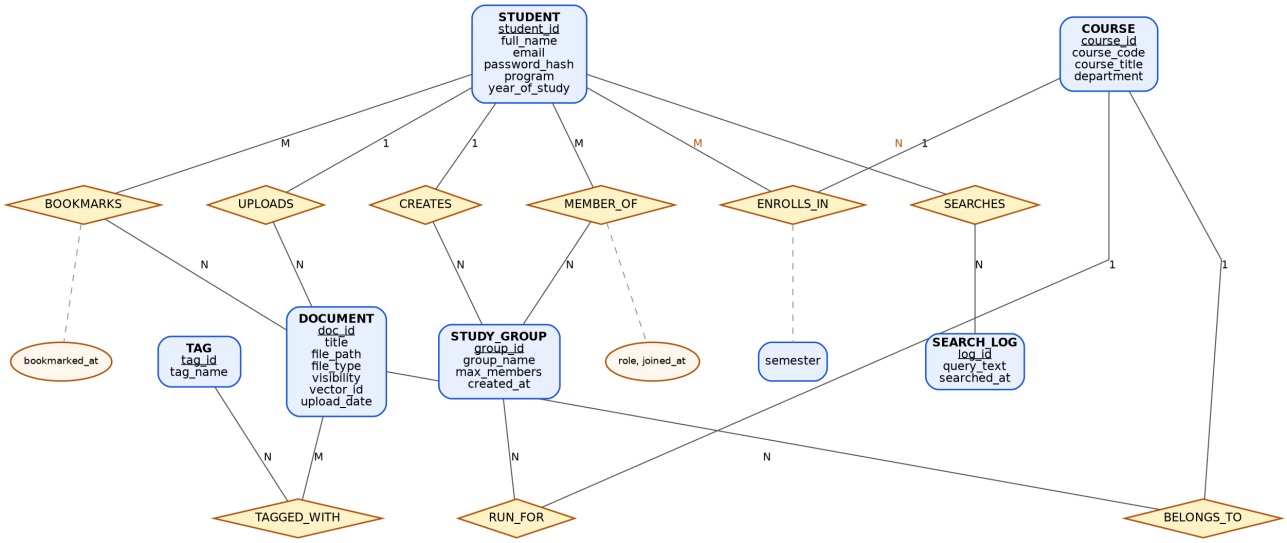
- Semantic retrieval (vector similarity search) replaces simple keyword `WHERE title LIKE` queries.
- Study-group matching logic combines relational data (course enrollment) with behavioural signals (search history, tags of bookmarked/uploaded documents) — a join across paradigms, not a single-table lookup.
- Access control (public / course-only / private visibility) governs what a search can even return.

Database paradigm justification (per project brief)

Data	Store	Why
Users, courses, enrollment, groups, membership, bookmarks, tags	**SQL — PostgreSQL**	Highly structured, relational integrity matters (a membership must reference a real student and group); needs transactions when a student joins/leaves a group.
Note/paper content embeddings	**Vector DB — pgvector / Qdrant (Option 2)**	Notes are unstructured text; meaning-based retrieval requires embedding similarity, not exact match. Each vector is referenced from `DOCUMENT.vector_id`, keeping the relational side authoritative for ownership/access control.

This week's deliverable covers the **SQL / relational** side only, per the Week 4 lab brief (ER diagram → relational schema → DDL → execution). The vector component will be documented and implemented in a later milestone.

2. Entity–Relationship Diagram



Rectangles = entities (primary key underlined). Diamonds = relationships, labelled with cardinality (1 / M / N). Dashed ovals = relationship attributes (attributes of the associative M:N relationships).

Entities & key attributes

Entity	Primary key	Other attributes
STUDENT	student_id	full_name, email, password_hash, program, year_of_study, created_at
COURSE	course_id	course_code, course_title, department
DOCUMENT	doc_id	title, file_path, file_type, visibility, vector_id, upload_date
TAG	tag_id	tag_name
STUDY_GROUP	group_id	group_name, max_members, created_at
SEARCH_LOG	log_id	query_text, searched_at

Relationships & cardinality

Relationship	Participating entities	Cardinality	Relationship attributes
ENROLLS_IN	STUDENT, COURSE	M : N	semester
UPLOADS	STUDENT, DOCUMENT	1 : N	—
BELONGS_TO	DOCUMENT, COURSE	N : 1 (optional)	—
TAGGED_WITH	DOCUMENT, TAG	M : N	—
BOOKMARKS	STUDENT, DOCUMENT	M : N	bookmarked_at
CREATES	STUDENT, STUDY_GROUP	1 : N	—
RUN_FOR	STUDY_GROUP, COURSE	N : 1	—
MEMBER_OF	STUDENT, STUDY_GROUP	M : N	role, joined_at
SEARCHES	STUDENT, SEARCH_LOG	1 : N	—

3. ER-to-Relational Mapping & Relational Schema

Standard mapping rules applied: every strong entity becomes a table with its PK; a 1:N relationship places the foreign key on the "N" side; every M:N relationship (with or without its own attributes) becomes a separate junction table whose primary key is the composite of the two participating foreign keys.

#	Table	Primary Key	Foreign Keys	Notes
1	STUDENT	student_id	—	Strong entity
2	COURSE	course_id	—	Strong entity
3	DOCUMENT	doc_id	student_id → STUDENT, course_id → COURSE (nullable)	FK from UPLOADS (1:N) & BELONGS_TO (N:1)
4	TAG	tag_id	—	Strong entity
5	STUDY_GROUP	group_id	course_id → COURSE, created_by → STUDENT	FK from RUN_FOR (N:1) & CREATES (1:N)
6	SEARCH_LOG	log_id	student_id → STUDENT	FK from SEARCHES (1:N)
7	ENROLLMENT	(student_id, course_id, semester)	student_id → STUDENT, course_id → COURSE	Junction for ENROLLS_IN (M:N)
8	DOCUMENT_TAG	(doc_id, tag_id)	doc_id → DOCUMENT, tag_id → TAG	Junction for TAGGED_WITH (M:N)
9	BOOKMARK	(student_id, doc_id)	student_id → STUDENT, doc_id → DOCUMENT	Junction for BOOKMARKS (M:N)
10	GROUP_MEMBERSHIP	(group_id, student_id)	group_id → STUDY_GROUP, student_id → STUDENT	Junction for MEMBER_OF (M:N)

Schema (shorthand notation), primary keys underlined, foreign keys italicised:

```
STUDENT(student_id, full_name, email, password_hash, program, year_of_study, created_at)
COURSE(course_id, course_code, course_title, department)
DOCUMENT(doc_id, title, file_path, file_type, visibility, vector_id, upload_date,
         student_id, course_id)
TAG(tag_id, tag_name)
STUDY_GROUP(group_id, group_name, max_members, created_at, course_id, created_by)
SEARCH_LOG(log_id, query_text, searched_at, student_id)
ENROLLMENT(student_id, course_id, semester)
DOCUMENT_TAG(doc_id, tag_id)
BOOKMARK(student_id, doc_id, bookmarked_at)
GROUP_MEMBERSHIP(group_id, student_id, role, joined_at)
```

4. DDL Statements

Target RDBMS: **PostgreSQL 16**. Full script below (also submitted separately as `schema.sql`).

```
-- ============================================================
-- ScholarMesh: Semantic Notes Discovery & Study Group Matching
-- SQL (Relational) Schema — DDL
-- Database: PostgreSQL
-- ============================================================

DROP TABLE IF EXISTS GROUP_MEMBERSHIP CASCADE;
DROP TABLE IF EXISTS BOOKMARK CASCADE;
DROP TABLE IF EXISTS DOCUMENT_TAG CASCADE;
DROP TABLE IF EXISTS ENROLLMENT CASCADE;
DROP TABLE IF EXISTS SEARCH_LOG CASCADE;
DROP TABLE IF EXISTS STUDY_GROUP CASCADE;
DROP TABLE IF EXISTS TAG CASCADE;
DROP TABLE IF EXISTS DOCUMENT CASCADE;
DROP TABLE IF EXISTS COURSE CASCADE;
DROP TABLE IF EXISTS STUDENT CASCADE;

-- ------------------------------------------------------------
-- 1. STUDENT  (strong entity)
-- ------------------------------------------------------------
CREATE TABLE STUDENT (
    student_id      SERIAL PRIMARY KEY,
    full_name       VARCHAR(100)  NOT NULL,
    email           VARCHAR(120)  NOT NULL UNIQUE,
    password_hash   VARCHAR(255)  NOT NULL,
    program         VARCHAR(80),
    year_of_study   SMALLINT      CHECK (year_of_study BETWEEN 1 AND 6),
    created_at      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- ------------------------------------------------------------
-- 2. COURSE  (strong entity)
-- ------------------------------------------------------------
CREATE TABLE COURSE (
    course_id       SERIAL PRIMARY KEY,
    course_code     VARCHAR(20)   NOT NULL UNIQUE,
    course_title    VARCHAR(150)  NOT NULL,
    department      VARCHAR(80)
);

-- ------------------------------------------------------------
-- 3. DOCUMENT  (strong entity; FKs from UPLOADS 1:N, BELONGS_TO N:1)
-- ------------------------------------------------------------
CREATE TABLE DOCUMENT (
    doc_id          SERIAL PRIMARY KEY,
    title           VARCHAR(200)  NOT NULL,
    file_path       VARCHAR(255)  NOT NULL,
    file_type       VARCHAR(20)   NOT NULL,
    visibility      VARCHAR(10)   NOT NULL DEFAULT 'private'
                        CHECK (visibility IN ('public','course','private')),
    vector_id       VARCHAR(64),          -- reference id of the embedding stored in the vector DB
    upload_date     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    student_id      INTEGER       NOT NULL REFERENCES STUDENT(student_id) ON DELETE CASCADE,
    course_id       INTEGER       REFERENCES COURSE(course_id) ON DELETE SET NULL
);

-- ------------------------------------------------------------
-- 4. TAG  (strong entity)
-- ------------------------------------------------------------
CREATE TABLE TAG (
    tag_id          SERIAL PRIMARY KEY,
    tag_name        VARCHAR(50)   NOT NULL UNIQUE
);
```

```
-- ------------------------------------------------------------
-- 5. STUDY_GROUP  (FKs from CREATES 1:N, RUN_FOR N:1)
-- ------------------------------------------------------------
CREATE TABLE STUDY_GROUP (
    group_id        SERIAL PRIMARY KEY,
    group_name      VARCHAR(100)  NOT NULL,
    max_members     SMALLINT      NOT NULL DEFAULT 6 CHECK (max_members > 0),
    created_at      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    course_id       INTEGER       NOT NULL REFERENCES COURSE(course_id) ON DELETE CASCADE,
    created_by      INTEGER       NOT NULL REFERENCES STUDENT(student_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 6. SEARCH_LOG  (FK from SEARCHES 1:N)
-- ------------------------------------------------------------
CREATE TABLE SEARCH_LOG (
    log_id          SERIAL PRIMARY KEY,
    query_text      VARCHAR(255)  NOT NULL,
    searched_at     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    student_id      INTEGER       NOT NULL REFERENCES STUDENT(student_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 7. ENROLLMENT  (M:N associative entity: STUDENT <-> COURSE)
-- ------------------------------------------------------------
CREATE TABLE ENROLLMENT (
    student_id      INTEGER       NOT NULL REFERENCES STUDENT(student_id) ON DELETE CASCADE,
    course_id       INTEGER       NOT NULL REFERENCES COURSE(course_id) ON DELETE CASCADE,
    semester        VARCHAR(20)   NOT NULL,
    PRIMARY KEY (student_id, course_id, semester)
);

-- ------------------------------------------------------------
-- 8. DOCUMENT_TAG  (M:N associative entity: DOCUMENT <-> TAG)
-- ------------------------------------------------------------
CREATE TABLE DOCUMENT_TAG (
    doc_id          INTEGER       NOT NULL REFERENCES DOCUMENT(doc_id) ON DELETE CASCADE,
    tag_id          INTEGER       NOT NULL REFERENCES TAG(tag_id) ON DELETE CASCADE,
    PRIMARY KEY (doc_id, tag_id)
);

-- ------------------------------------------------------------
-- 9. BOOKMARK  (M:N associative entity: STUDENT <-> DOCUMENT)
-- ------------------------------------------------------------
CREATE TABLE BOOKMARK (
    student_id      INTEGER       NOT NULL REFERENCES STUDENT(student_id) ON DELETE CASCADE,
    doc_id          INTEGER       NOT NULL REFERENCES DOCUMENT(doc_id) ON DELETE CASCADE,
    bookmarked_at   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (student_id, doc_id)
);

-- ------------------------------------------------------------
-- 10. GROUP_MEMBERSHIP  (M:N associative entity: STUDENT <-> STUDY_GROUP)
-- ------------------------------------------------------------
CREATE TABLE GROUP_MEMBERSHIP (
    group_id        INTEGER       NOT NULL REFERENCES STUDY_GROUP(group_id) ON DELETE CASCADE,
    student_id      INTEGER       NOT NULL REFERENCES STUDENT(student_id) ON DELETE CASCADE,
    role            VARCHAR(10)   NOT NULL DEFAULT 'member' CHECK (role IN ('admin','member')),
    joined_at       TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (group_id, student_id)
);

-- ------------------------------------------------------------
-- Supporting indexes (beyond PK/UNIQUE auto-indexes)
-- ------------------------------------------------------------
CREATE INDEX idx_document_student   ON DOCUMENT(student_id);
CREATE INDEX idx_document_course    ON DOCUMENT(course_id);
CREATE INDEX idx_document_visibility ON DOCUMENT(visibility);
CREATE INDEX idx_studygroup_course  ON STUDY_GROUP(course_id);
```

```
CREATE INDEX idx_searchlog_student  ON SEARCH_LOG(student_id);
CREATE INDEX idx_tag_name           ON TAG(tag_name);
```

5. Execution

The script was executed against a live PostgreSQL 16 instance (`psql -d scholarmesh -f schema.sql`). All 10 `CREATE TABLE` and 6 `CREATE INDEX` statements executed with no errors:

```
$ psql -d scholarmesh -f schema.sql
CREATE TABLE  (x10)
CREATE INDEX  (x6)
```

Verification — table listing (`\dt`)

```
             List of relations
 Schema |       Name       | Type  |  Owner
--------+------------------+-------+----------
 public | bookmark         | table | postgres
 public | course           | table | postgres
 public | document         | table | postgres
 public | document_tag     | table | postgres
 public | enrollment       | table | postgres
 public | group_membership | table | postgres
 public | search_log       | table | postgres
 public | student          | table | postgres
 public | study_group      | table | postgres
 public | tag              | table | postgres
(10 rows)
```

Verification — DOCUMENT table structure (`\d document`)

```
                                      Table "public.document"
   Column    |            Type             | Collation | Nullable |                Default
-------------+-----------------------------+-----------+----------+---------------------------------------
 doc_id      | integer                     |           | not null | nextval('document_doc_id_seq'::regclass)
 title       | character varying(200)      |           | not null |
 file_path   | character varying(255)      |           | not null |
 file_type   | character varying(20)       |           | not null |
 visibility  | character varying(10)       |           | not null | 'private'::character varying
 vector_id   | character varying(64)       |           |          |
 upload_date | timestamp without time zone |           | not null | CURRENT_TIMESTAMP
 student_id  | integer                     |           | not null |
 course_id   | integer                     |           |          |
Indexes:
    "document_pkey" PRIMARY KEY, btree (doc_id)
    "idx_document_course" btree (course_id)
    "idx_document_student" btree (student_id)
    "idx_document_visibility" btree (visibility)
Check constraints:
    "document_visibility_check" CHECK (visibility::text = ANY (ARRAY['public'::character varying, 'course'::character varying, 'private'::character
varying]::text[]))
Foreign-key constraints:
    "document_course_id_fkey" FOREIGN KEY (course_id) REFERENCES course(course_id) ON DELETE SET NULL
    "document_student_id_fkey" FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE
Referenced by:
    TABLE "bookmark" CONSTRAINT "bookmark_doc_id_fkey" FOREIGN KEY (doc_id) REFERENCES document(doc_id) ON DELETE CASCADE
    TABLE "document_tag" CONSTRAINT "document_tag_doc_id_fkey" FOREIGN KEY (doc_id) REFERENCES document(doc_id) ON DELETE
CASCADE
```

Verification — GROUP_MEMBERSHIP table structure (`\d group_membership`)

```
                  Table "public.group_membership"
   Column   |          Type          | Collation | Nullable |          Default
------------+------------------------+-----------+----------+----------------------------
 group_id   | integer                |           | not null |
 student_id | integer                |           | not null |
```

```
 role       | character varying(10)       |           | not null | 'member'::character varying
 joined_at  | timestamp without time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "group_membership_pkey" PRIMARY KEY, btree (group_id, student_id)
Check constraints:
    "group_membership_role_check" CHECK (role::text = ANY (ARRAY['admin'::character varying, 'member'::character varying]::text[]))
Foreign-key constraints:
    "group_membership_group_id_fkey" FOREIGN KEY (group_id) REFERENCES study_group(group_id) ON DELETE CASCADE
    "group_membership_student_id_fkey" FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE
```

✓ All primary keys, foreign keys, and CHECK constraints were created exactly as designed in the schema mapping — confirmed above for the two most complex tables (one strong entity with two FKs, one M:N junction with a composite PK and a CHECK constraint).

6. Next Steps (beyond Week 4 scope)

- DML: seed data, and demonstrate JOIN / GROUP BY / subquery operations across the schema above.
- Vector DB integration: choose embedding model, set up pgvector/Qdrant collection, link via `vector_id`.
- Advanced features (min. 4 required): candidates are Views (e.g. a "student dashboard" view), a Trigger to cap `GROUP_MEMBERSHIP` at `max_members`, a stored procedure for group-matching, and full-text/vector search.
- Next.js application layer with authentication and role-based access (Student / Group-Admin roles).